



RoadStar Hackathon Build Blueprint: Unified Dispatch, Automation, and Open-Source Reuse

What you should actually build

After putting together the **project brief**, the **workbook RoadStar gave you**, the **judge's messages**, the **current commercial market**, **current freight-tech trends**, and **open-source projects**, I think the ambiguity is basically gone.

Your project should **not** be “invent a brand-new category of trucking software.”

It should be:

A RoadStar-specific unified dispatch operating system that first reproduces the dispatcher's existing manual workflow, then adds an intelligent decision layer that automates the painful parts.

For the rest of this report I will call the concept **RoadStar DispatchOS**, but the name obviously can change.

The judge's messages are unusually clear on this. The workflow needs to exist manually first: dispatcher sees loads, drivers, trucks and trailers; dispatcher can put loads on trucks manually; then your automated system does the same job faster. The project brief independently asks for the same basic architecture: a web dispatcher dashboard, mapping, track-and-trace, geofences and detention, an independent telemetry simulator, HOS logic, load matching, driver interface, and consolidation of TMS/load-board/ELD information.

So your fundamental product loop should be:

Loads arrive → dispatcher can assign them manually → system evaluates every feasible driver/truck/load combination → “Plan My Morning” generates an optimized dispatch plan → dispatcher reviews why each choice was made → accepts/overrides it → drivers receive assignments → trucks move on a live map → HOS/geofences/traffic/detention update continuously → system warns or re-plans when reality changes.

That is a much stronger framing than “we built a dashboard.”

The most important strategic decision

Do **not** make AI chat the centre of the product.

Make the **optimizer** the centre of the product.

AI can sit around it for document parsing, explanations, natural-language querying and eventually autonomous workflows. But which truck should take which load is a constrained optimization problem, not primarily an LLM problem. This is also where the commercial market is going: Optimal Dynamics' current platform explicitly combines mathematical decision optimization with automation, including network-aware dispatch, re-optimization and, in its 2026 Scale product, autonomous agents that can source freight across load boards, email and EDI. ¹

The ideal hackathon experience is therefore:

Optimizer decides → software explains → dispatcher approves → automation executes.

That is also almost exactly what your judge described: preserve the manual process, then layer automation on top.

Your supplied dataset is actually excellent for this

I inspected `Hackathon_Data.xlsx` directly. You already have enough RoadStar-specific information to build something far more convincing than a generic prototype:

Data in your workbook	What you have
Order rows	4,031
Unique bill numbers	3,938
Historical dispatch legs	10,479
Unique dispatch trips	2,628
Unique drivers	131
Trucks	131
Trailers	425
Loaded dispatch legs	6,756
Empty dispatch legs	3,723
Loaded recorded distance	~1.373 million
Empty recorded distance	~222,635
Share of recorded distance that is empty	~13.95%

That last figure is particularly valuable. Using the supplied `LS_LEG_DIST` field, about **13.95% of recorded historical dispatch distance is on legs marked empty**. You therefore have a RoadStar-specific baseline for the deadhead story rather than relying on a generic trucking statistic.

The workbook contains many of the exact features an optimizer needs: pickup and delivery windows, origins/destinations, weights, pallet counts, dry-van/reefer/flatbed load types, temperature-control

requirements, planned times, historical dispatch legs, driver locations, current duty information, remaining hours, projected HOS violation fields, assigned power units, trailers and trailer capacities.

There is even a potentially interesting **data-quality/operational exception** hiding in the file: the orders include **218 Flatbed loads**, while the trailer master sheet contains Dry Van and Reefer trailer types but no Flatbed type. That does *not* prove RoadStar lacks flatbeds—it could reflect leased assets, incomplete master data, historical data or a modelling choice—but your system should detect exactly this kind of mismatch rather than silently force an assignment:

 **No compatible trailer in current asset master — manual review required.**

That directly speaks to the brief's bonus category of discovering hidden operational/software edge cases.

The project is really four systems in one

Think of DispatchOS as four interconnected layers:

System of record

Loads + drivers + trucks + trailers + stops + assignments + HOS + telemetry + documents.

System of visibility

Maps, truck locations, breadcrumbs, current assignments, ETA, speed, detention, alerts.

System of decision

Load matching, HOS feasibility, trailer compatibility, deadhead reduction, multi-load planning, future-position/backhaul logic.

System of action

Dispatcher approval, driver dispatch, reassignments, mobile workflow, notifications and eventually real integrations.

That architecture mirrors what the commercial market has discovered is valuable: Samsara provides extensive telematics/ELD/dispatch capabilities, while integrations such as Samsara-McLeod combine GPS and HOS from the telematics side with TMS dispatch workflows so dispatchers do not have to jump between systems. ²

And you have very little time to wander. The official RoadStar schedule currently shows **code freeze at 2:00 p.m. on Sunday, September 13**, followed immediately by live presentations. September 9 is explicitly labelled a buffer/prep day and September 12 as final preparation. ³

So scope discipline matters enormously.

The complete feature checklist and scope

I would separate your backlog into three groups: **must work**, **high-scoring differentiation**, and **wow features**. The most important rule is that nothing in the third category is allowed to break or delay the first.

Priority	Capability	What it should actually do	Build or reuse?
P0	Unified data model	Drivers, trucks, trailers, loads, stops, trips and HOS all reference one source of truth	Build
P0	Load board	Unassigned / assigned / in-transit / completed loads with filters and details	Build; borrow TMS UX
P0	Driver & asset board	Driver location, duty/HOS, assigned truck, trailer, status and next availability	Build
P0	Manual dispatch	Dispatcher can manually pair load → driver/ truck/trailer, change it, unassign it and override system recommendations	Essential because judge asked for it
P0	Feasibility checker	Prevent or flag impossible assignments before dispatch	Build + optimization library
P0	Morning Auto-Plan	One button plans multiple unassigned loads against available fleet simultaneously	OR-Tools/VROOM
P0	Map	Southern Ontario fleet map, routes, pickup/ drop points, truck locations, satellite toggle	MapLibre/Mapbox
P0	Real-time simulator	Trucks move along routes; GPS, speed, distance and HOS change over time	Build thin simulator
P0	Breadcrumb history	Show path, speed and distance travelled	Build; Traccar inspiration
P0	HOS checks	Prevent/flag assignments predicted to violate practical Canadian HOS limits	Build + supplied HOS data
P0	Geofences	Detect facility arrival/departure and persist timestamps	Turf/PostGIS
P0	Detention engine	Calculate dwell, free time and billable detention automatically	Build
P0	Driver interface	Loads, stops, route, accept/acknowledge assignment and duty/HOS status	Responsive PWA
P0	Live synchronization	Dispatcher sees driver/simulator changes without refreshing	WebSocket/SSE

Priority	Capability	What it should actually do	Build or reuse?
P1	Explainable recommendations	"Why truck 104?" and "Why not truck 112?"	Build
P1	Exceptions inbox	HOS risk, late pickup, equipment mismatch, missing data, detention, route incident	Build
P1	Automatic re-plan	Re-evaluate assignments when delay/HOS/ location changes	Optimizer
P1	Deadhead/backhaul assistant	Suggest next freight after delivery rather than optimizing one load in isolation	Optimizer
P1	Ontario 511 integration	Road closures, incidents, construction, conditions and cameras	Official Ontario API
P1	Historical replay	Replay supplied RoadStar history and compare baseline vs optimized scenario	Build
P1	KPI/ROI dashboard	Empty km, coverage, HOS risk, late-risk, detention captured, planning time	Build
P1	Integration adapters	Clearly show TMS, ELD, map and traffic providers as replaceable connections	Build
P2	Multi-stop LTL consolidation	Combine compatible smaller shipments and optimize stop sequence	OR-Tools/VROOM
P2	3D trailer loading	Render where freight physically goes inside trailer	xflp + Three.js/ front end
P2	Stop-aware loading	First-delivered freight remains accessible; last-delivered freight can go deeper	xflp
P2	Weight/balance visualizer	Check weight/capacity and approximate placement/balance constraints	xflp/custom
P2	AI document intake	Parse a rate confirmation/BOL into a new load	LLM/OCR
P2	Dispatcher copilot	Ask "What loads are at risk?" or "Why was John not assigned?"	LLM over structured data
P2	POD/document flow	Driver photo/document goes back to the load	Build
Later	Accounting/payroll	Invoices, settlements, payroll	Do not spend hackathon time here

Priority	Capability	What it should actually do	Build or reuse?
Later	Full maintenance suite	PM schedules, DVIR repairs, parts etc.	Do not spend hackathon time here
Later	Certified ELD replacement	Official compliant ELD hardware/software	Absolutely not a week-one scope

The brief explicitly requires most of the P0 operational layer: live/simulated telematics, track-and-trace, satellite mapping, geofences, detention, HOS, load matching and a mobile/responsive driver interface.

filecite turn0file0

What “manual dispatch” should look like

This should resemble real dispatcher software enough that the judge immediately recognizes it.

A good main screen is:

Left: unassigned loads

Centre: truck/driver plan or timeline

Right: map / selected entity

Drawer: load, driver, truck and HOS details

A load row might show:

RS-4521 | Milton → London | 10:00–12:00 | Dry Van | 31,200 lb | 18 pallets

An available truck/driver row might show:

Truck 84 | Driver D113 | Mississauga | 8h42 driving | Dry Van | Available 08:30

The dispatcher can click/drag one onto the other.

Then the exact same interface has:

🌟 Auto Plan Morning

When clicked, the software should **not immediately mutate everything**. It should open a proposal:

Load	Recommended unit	Deadhead	Pickup ETA	HOS after trip	Why
4521	D113 / T84	23 km	09:28	5h 12m	closest feasible dry van + strong downstream position
4528	D077 / T31	31 km	10:04	3h 44m	reefer required

Load	Recommended unit	Deadhead	Pickup ETA	HOS after trip	Why
4530	—	—	—	—	no compatible asset / HOS

Buttons:

Apply All · Review Individually · Reject Plan

This single interaction is likely the centrepiece of your entire demo.

Build an exceptions system, not just a pretty dashboard

The software should actively identify things a dispatcher would otherwise discover by bouncing between tabs.

Examples from your actual RoadStar problem:

HOS conflict: Driver can reach pickup but cannot legally complete delivery.

Equipment conflict: Refrigerated freight assigned to a dry van.

Data conflict: Flatbed load but no matching trailer in current asset master.

Time-window conflict: Nearest driver arrives after pickup window.

Dock/HOS collision: Driver arrived legally but remaining on-duty clock will expire while waiting.

Road disruption: 401 incident changes ETA enough to threaten delivery.

Detention opportunity: Truck has been inside shipper geofence for 1h45m; billable threshold approaching.

The brief explicitly calls out the “driver runs out of HOS sitting at a dock” and highway-closure cases as examples of valuable hidden problems. [filecite turn0file0](#)

That tells you something important about the judges:

Showing that your software understands the ugly edge cases may score better than adding ten generic dashboard charts.

What is already in the market and what to borrow

Your original concern was correct: **almost every individual feature exists somewhere already.**

That is not a problem anymore.

It is actually evidence that these workflows matter.

The opportunity is to combine them around **RoadStar's workflow and data** while making the decision-making substantially more automated.

Product/category	What exists today	What I would borrow
Samsara	Fleet GPS, HOS/ELD, routing, dispatch, geofences, asset tracking, diagnostics and driver workflows within its Connected Operations platform. ⁴	Map/asset UX, HOS presentation, live telemetry, exception alerts
McLeod LoadMaster	TMS dispatch combined through integrations with ELD/GPS; McLeod's Samsara integration brings GPS/HOS into LoadMaster and supports end-to-end dispatch and driver communication. ⁵	Traditional dispatcher workflow and "system of record" model
Alvys	Modern all-in-one TMS covering dispatch, load management, driver mobile, compliance, accounting, analytics/API, marketplace and 120+ integrations. ⁶	Clean unified-TMS idea and integrations screen
Alvys + Samsara	Telematics/HOS integration, dispatched routes, automatic geofence-derived stop arrival/departure and driver documents returned to the load. ⁷	The exact TMS↔ELD data-flow RoadStar is describing
Optimal Dynamics	Automated driver-to-load decisions, HOS/location/home-time-aware recommendations, network optimization and real-time re-optimization. ⁸	Your automated dispatch engine
Optimal Dynamics Atlas	Explains why a driver-load pairing was or was not recommended and shows availability/ETA reasoning. ⁹	Your explainability layer
Optimal Dynamics Source/Scale	Evaluates load-board freight in context of existing network needs; Scale extends that into proactive agentic sourcing/negotiating/procurement. ¹⁰	Your future backhaul/network-aware idea
Motive integrations	Motive APIs expose vehicle/driver location and HOS that external dispatch products use for automation. ¹¹	Another future ELD adapter
Samsara Driver App	HOS, routing, inspections and dispatcher communication in a driver-facing app. ¹²	Driver PWA workflow

The most revealing existing product for your problem is probably not one individual TMS but the **McLeod + Samsara integration**.

McLeod describes bringing Samsara GPS and HOS directly into LoadMaster so dispatchers can find nearby drivers with available hours; the expanded integration lets them dispatch, communicate with drivers and see trailers without leaving the TMS. ¹³

That is almost word-for-word the problem your judge explained to you.

So yes:

You are building something the industry already knows is useful.

Your challenge is to show that RoadStar can have a more cohesive, RoadStar-specific version with intelligent automation rather than stitching multiple subscription products together. That is also consistent with the hackathon brief's explicit goal of demonstrating that integrated intelligent dispatch dashboards can now be created rapidly and affordably. ¹⁴

Where the market is moving right now

The frontier is no longer merely “put everything on one dashboard.”

It is moving from:

visibility → recommendation → optimization → controlled autonomy.

Optimal Dynamics now markets real-time dispatch automation and network-wide decision-making, and its 2026 Scale release takes the next step by having agents source and secure freight according to an optimization engine's view of the carrier's entire network. ¹ Alvys is similarly embedding automated load creation, agent workflows and real-time AI alerts directly into its TMS rather than putting AI in a separate application. ¹⁴

There is also already Canadian movement in this direction. Optimal Dynamics announced its first Canadian expansion with Challenger Motor Freight in late 2025, specifically citing automated load acceptance/dispatch, Canadian HOS considerations and scenario simulation among the use cases. ¹⁵

This gives you an excellent innovation narrative:

“We're bringing the emerging decision-automation model of enterprise freight technology into a lightweight RoadStar-specific dispatcher platform.”

The differentiation I would actually pursue

Do not try to claim:

“No one has automated dispatch before.”

That is demonstrably false.

Claim:

“We unify RoadStar's dispatch, load, asset, HOS, tracking, detention and external-road-state data into one operational state, then use that common state to automate the dispatcher's highest-friction decisions.”

And make your distinctive pieces:

Batch planning, not one-load-at-a-time matching.

The dispatcher has 20 loads and 20 trucks. Optimize them together.

Look-ahead, not nearest-truck matching.

The closest driver now may be the worst driver after delivery.

Explainability.

Every recommendation tells the dispatcher why it happened and why alternatives lost.

Live re-optimization.

The morning plan isn't frozen. HOS, traffic, detention and new loads cause it to evolve.

Historical proof using RoadStar's data.

Do not just say “we reduce deadhead.” Demonstrate baseline versus simulated optimized plan.

Southern Ontario intelligence.

Ontario 511 has a developer API exposing traffic events, construction, cameras, road conditions, truck rest areas, inspection stations and other transportation data; its documented throttle is ten calls per 60 seconds.

16

That last one is especially good because it is local, useful, free/public, and directly applicable to the 401/403/400 corridors the brief emphasizes. [filecite turn0file0](#)

Imagine the map showing:

 401 WB collision – lane closure

Truck 84 ETA +27 min

HOS completion margin falls from 1h06m → 39m

Re-optimize?

That feels like an operational product rather than a school dashboard.

The GitHub and open-source reuse map

This is where you can save a **lot** of time.

The correct strategy is **not to find one giant TMS and change the logo.**

That creates licence problems, unfamiliar architecture, dependency problems and a massive surface area to debug.

Instead:

Build a thin RoadStar-specific shell and borrow proven specialist components for the difficult algorithms.

Here are the repositories I would actually consider.

Repository	Use in your project	Licence / concern	Verdict
<code>google/or-tools</code>	Global load↔driver assignment, constraint programming, routing/ time-window problems	Apache-2.0 in current repo metadata; mature Google operations-research toolkit.	USE
<code>VRROOM-Project/vroom</code>	Fast VRP, capacities, time windows, pickup/delivery and multi-stop planning	BSD-2-Clause; current project supports CVRP, VRPTW, pickup/delivery, skills, capacity, working hours and breaks. ¹⁷	USE or OR-Tools alternative
<code>traccar/traccar</code>	GPS tracking, histories, geofences, alerts and a REST-backed tracking architecture	Apache-2.0; supports real-time GPS, geofences, reports and notifications. ¹⁸	BORROW / optionally run
<code>traccar/traccar-web</code>	Great reference for a real fleet tracking UI	React + Material UI + MapLibre; Apache-2.0. ¹⁹	STRONG UI REFERENCE
<code>maplibre/maplibre-gl-js</code>	Interactive fleet map	BSD-3-Clause, GPU-accelerated web map rendering. ²⁰	USE
<code>Turfjs/turf</code>	Point-in-polygon, distance and other geospatial/ geofence calculations	MIT. ²¹	USE
<code>graphhopper/graphhopper</code>	Self-hosted OSM routing, distances and times	Apache-2.0; can run as Java library/server. ²²	USE if avoiding commercial routing API
<code>hschneid/xflp</code>	Your truck-loading Tetris engine	MIT; LIFO, unloading sequence, max weight, bearing weight, stacking groups and two-axle load constraints. ²³	BEST 3D SOLVER FIT

Repository	Use in your project	Licence / concern	Verdict
<code>skjolber/3d-bin-container-packing</code>	Alternative 3D packing engine and ready-made Three.js visualizer	Apache-2.0; obstacles and customizable placement/manifest controls. ²⁴	VERY GOOD FALLBACK
<code>enzoruiz/3dbinpacking</code>	Very quick Python 3D packing prototype	Simple Python API supporting bin dimensions and maximum weight. ²⁵	FASTEST 3D FALLBACK
<code>TimefoldAI/timefold-solver</code>	Advanced Java/Kotlin constraint optimisation	Apache-2.0 Community Edition; actively targets vehicle routing and resource allocation. ²⁶	GOOD, but OR-Tools likely faster for your stack
<code>fleetbase/fleetbase</code>	Full logistics-platform architecture, fleet map, order board, API/webhooks	AGPL-3.0 by default; modified network-service deployments trigger source-availability obligations unless commercially licensed. ²⁷	STUDY IT; DON'T CASUALLY FORK IT
<code>suxrobGM/logistics-app</code> / LogisticsX	Astonishingly close open project: trucking TMS, AI dispatcher, HOS, Mapbox, driver app, load boards, approval workflow	PolyForm Noncommercial. Repo says business use/hosting needs a commercial licence. ²⁸	ABSOLUTELY STUDY; BE CAREFUL COPYING CODE

The repository you should study tonight: LogisticsX

`SuxrobGM/logistics-app` is probably the closest GitHub project I found to the thing you have been asked to make.

Its current README describes a trucking fleet management system where dispatchers create loads, assign drivers, search load boards and monitor deliveries; drivers use a mobile application; an AI dispatcher evaluates trucks and loads, checks HOS, plans trips and proposes assignments that a dispatcher accepts or rejects. It also includes multi-stop routes, live driver maps, Mapbox, Samsara/Motive/Geotab HOS integration concepts, maintenance, documents, chat and load-board integrations. ²⁸

That makes it **phenomenal competitive research**.

Study:

- its information architecture;
- how it separates loads/trips/fleet;
- its AI approval flow;
- its driver application screens;
- how integrations are structured;
- how pending decisions are presented.

But do **not** blindly clone it. Its repository explicitly states PolyForm Noncommercial licensing and says running it in a business/hosting it for others requires a commercial licence. ²⁸

For a hackathon, treat it as an **architecture and UX reference unless your team has confirmed that its licence permits your intended use.**

The repository I would not use as your foundation: Fleetbase

Fleetbase is also impressively complete: order boards, live fleet maps, tracking, zones, REST APIs, sockets/webhooks, telematics and native apps. ²⁷

But it is a much broader logistics operating system and its default AGPL-3.0 licence has important obligations for modified network-hosted versions. Fleetbase itself offers separate commercial licensing for organizations wanting proprietary modifications. ²⁷

In a five-day sprint, I would inspect it for patterns rather than spend precious time learning and remodelling the whole framework.

The combination I would actually use

My preferred hackathon reuse stack is:

Problem	Component
Dispatch/global assignment	Google OR-Tools
Route/multi-stop optimization alternative	VROOM
Browser map	MapLibre GL JS
Actual route/travel-time matrix	Mapbox API if credits are supplied , otherwise GraphHopper/OSRM
Geofence math	Turf.js
GPS/tracking architecture	Borrow from Traccar
3D truck-loading solver	xflp
3D rendering	Three.js or xflp/skjolber output rendered in your own front end
Full TMS UX inspiration	LogisticsX + Alvys + McLeod
AI dispatch inspiration	Optimal Dynamics + LogisticsX

Google's OR-Tools documentation already demonstrates vehicle routing with time windows and the ability to model cumulative travel time, service windows and resource constraints, exactly the family of problems your dispatcher needs to solve. ²⁹

VROOM is attractive if you want a purpose-built routing service: its current open-source model supports capacities, time windows, priorities, vehicle skills, driver breaks and heterogeneous vehicles, and can run over OSRM, Valhalla or another custom cost matrix. ³⁰

Reuse code, not responsibility

One final warning: an open-source library solving “vehicle routing” does **not** mean it automatically understands RoadStar.

Your value is in translating RoadStar's business rules into constraints.

OR-Tools can find an optimum.

It cannot know on its own that:

- this load needs a reefer;
- this driver has 3h18 left;
- this pickup closes at noon;
- the driver is currently 27 minutes from the shipper;
- that trailer already contains another shipment;
- the second load causes a downstream HOS failure;
- the closest truck will create an awful empty return;
- a 401 closure invalidated the old ETA.

That domain layer is the software you are actually building.

Architecture, optimization logic, and the 3D “Tetris” idea

I would build a relatively boring architecture underneath a very impressive interface.

Boring is good in a hackathon.

Recommended technical stack

Layer	Recommended choice	Why
Web	Next.js / React / TypeScript	Fast UI development
Styling/ components	Tailwind + a component library	Avoid designing everything manually
Map	MapLibre GL JS	Open-source interactive mapping. ²⁰
Backend	FastAPI / Python	Extremely fast API development + easy OR-Tools use

Layer	Recommended choice	Why
Database	PostgreSQL / Supabase Postgres	Relational data fits dispatch very naturally
Optimization	OR-Tools Python	Main assignment/constraint engine
Routing	Mapbox Matrix / Directions or GraphHopper	Travel times and geometry
Real time	WebSockets or Server-Sent Events	Live simulator/dashboard
Geospatial	Turf.js and/or PostGIS	Geofence logic
Simulator	Separate Python process/service	Brief explicitly asks for separation
Driver app	Responsive PWA	Much faster than two native apps
3D solver	xflp sidecar	Real trucking-specific constraints
AI	LLM API only for language/document/ explanation layer	Keep decisions deterministic

The official event site says teams are being given developer tools/APIs/SDKs and mobile emulators, so use whatever sponsored mapping/cloud resources are available rather than unnecessarily self-hosting everything during the sprint. ³¹

The data architecture

Do not preserve the Excel structure as your application structure.

Normalize it roughly into:

```

Driver
Truck
Trailer
Load
Stop
Trip
Assignment
DriverClock
TelemetryPoint
Geofence
GeofenceEvent
DetentionEvent
RoadIncident
Recommendation
RecommendationReason

```

That last pair matters.

Store every automated decision.

For example:

```
Recommendation #A921
Load: RS-4521
Driver: D113
Truck: T084

Status: Recommended

deadhead_km          23.1
pickup_slack_min      52
driving_hours_after   5.2
on_duty_hours_after   4.5
trailer_compatible    true
temperature_compatible true
backhaul_score        0.81
```

Then your UI turns those facts into:

Why this truck?

D113 can reach the pickup 52 minutes before the appointment, has sufficient HOS, already has a compatible dry-van unit, and leaves the fleet in a stronger London-area position after delivery.

That explanation can be generated without an LLM at all.

An LLM may make the wording nicer, but the **underlying reason should come from structured solver output**.

That gives you something extremely valuable in dispatch automation:

trust.

Optimal Dynamics has itself made explainability a product feature: its Atlas tool lets dispatchers investigate why a given driver-load assignment was—or was not—recommended and inspect driver availability and projected execution. ⁹

How the load-matching engine should work

Do this in two stages.

First: eliminate impossible assignments.

For each driver/truck/trailer and load combination:

```
Is driver active?
Is truck available?
Is compatible trailer available?
Does equipment type match?
Does reefer requirement match?
Does load fit trailer weight capacity?
Can driver reach pickup window?
Can driver complete required driving within HOS?
Can pickup + service + delivery fit on-duty window?
Does route finish before delivery deadline?
Are required load restrictions satisfied?
```

Anything failing is not merely “low score.”

It is **infeasible**.

That distinction is important.

Second: optimize among feasible choices.

The objective can conceptually be:

$$\text{Minimize } w_d D + w_t T + w_l L + w_h H - w_b B - w_u U$$

Where:

- D = deadhead cost/distance,
- T = excess travel/time,
- L = lateness risk,
- H = HOS exhaustion risk,
- B = downstream/backhaul value,
- U = utilization/network value.

The weights should live in configuration rather than being buried in source code.

Your UI could even make this visible:

Optimization goal

```
Deadhead 40% | On-time 30% | HOS buffer 20% | Future positioning 10%
```

That turns the judge's exact question about trade-offs into a feature.

Do not use a greedy “nearest truck” algorithm as your final optimizer

Suppose:

Truck A is 10 km from Load 1.

Truck B is 30 km from Load 1.

A naive system chooses A.

But perhaps Load 1 finishes in London, where Truck A will then have nothing, while Truck B has a natural London→Kitchener load immediately after.

A global planner should evaluate:

current position → Load A → potential Load B

rather than just:

current position → Load A

That **look-ahead** is one of the strongest ways you can differentiate a real optimizer from a distance-sort button.

Optimal Dynamics' current product direction explicitly emphasizes network-wide consequences and sourcing freight that improves future utilization instead of evaluating every load in isolation. ¹⁰

HOS should be a hard operational constraint

For driving south of 60°N, current federal regulations include the familiar 13-hour driving and 14-hour on-duty limits, a 16-hour elapsed-time restriction around the required core rest period, 10 hours of daily off-duty time, and Cycle 1/2 accumulated-hour limits. ³²

For the hackathon, however, **do not attempt to rebuild an entire certified Canadian ELD rules engine.**

Your workbook already supplies fields such as remaining hours, duty status and projected HOS violation information.

Use those as your operational inputs and implement the key limits relevant to dispatch feasibility.

Your status display could look like:

D113 — AVAILABLE

Driving	8h 42m remaining
On Duty	9h 16m remaining

```
Elapsed      10h 08m remaining
Cycle       27h 31m remaining
```

Projected trip:

```
Deadhead      0h 31m
Loaded drive   2h 14m
Expected dock  1h 00m
```

Post-trip HOS margin: 5h 57m

✓ FEASIBLE

Much better than simply displaying `"REMAINING_HOURS = 8.7"`.

The integration architecture is itself a strong selling point

Don't hardwire everything to your simulator.

Use interfaces like:

```
TelematicsProvider
  SimulatorProvider
  SamsaraProvider
  MotiveProvider

LoadProvider
  RoadStarExcelProvider
  TMSProvider
  LoadBoardProvider

RoutingProvider
  MapboxProvider
  GraphHopperProvider

TrafficProvider
  Ontario511Provider
```

Today:

```
TelematicsProvider = Simulator
```

Tomorrow:

```
TelematicsProvider = Samsara
```

Samsara currently provides REST APIs and webhooks across its major product areas, including telematics and connected-driver systems, and its compliance APIs expose HOS/RODS functionality. ³³ Motive similarly exposes vehicle-location data and APIs used by external dispatch integrations for location/HOS workflows. ¹¹

This means your pitch becomes:

“This isn't a fake dashboard tied to our simulator. The simulator implements the same provider contract that a real Samsara/Motive integration would implement.”

That is a very strong engineering point.

Build Ontario 511 into that architecture

The Ontario 511 API is almost perfect for your particular hackathon. It exposes events, construction, cameras, road conditions, alerts, truck rest areas and inspection stations through REST resources. ³⁴

Use it initially as an **incident overlay**:

```
Ontario511 Incident
401 Westbound
Severity: High
Affected route: Truck 84
Original ETA: 11:16
Updated ETA: 11:43
Delivery deadline: 11:30

△ Delivery risk
[Re-route] [Re-optimize assignment]
```

If you have time, cameras can be a wow factor. Ontario's API exposes camera coordinates and metadata, and provincial traffic-camera data is explicitly available through Ontario 511. ³⁵

How the real-time simulator should work

Do not over-engineer it.

For every active trip:

1. Get route geometry.
2. Place truck at starting coordinate.
3. Every simulated interval, interpolate forward along route.
4. Update: `lat`, `lng`, `speed`, `odometer`, `driving time`, `on-duty time`.
5. Send telemetry event.
6. Backend checks geofences.

7. Frontend receives WebSocket update.
8. Occasionally inject: traffic slowdown, dock wait, HOS event or closure.

Use an accelerated clock.

For example:

1 real second = 2 simulated minutes

Then your live demo can show a two-hour detention event without making judges wait two hours.

Geofencing and detention

This should be database-driven, not a cosmetic timer.

State machine:

```
OUTSIDE FACILITY
    ↓ crosses boundary
ARRIVED
    ↓ remains inside
DWELLING
    ↓ exceeds free time
DETENTION
    ↓ leaves boundary
DEPARTED
```

Persist:

```
arrived_at
departed_at
total_dwell_minutes
free_minutes
billable_minutes
configured_rate
estimated_charge
```

The brief specifically treats automated geofence timestamps and post-two-hour FTL detention calculations as a critical requirement. [filecite turn0file0](#)

A brilliant enhancement would be alerts at:

90 minutes: approaching free-time limit

120 minutes: detention started

departure: evidence package ready

Now, your 3D truck-loading “Tetris” idea

This is genuinely good.

But it needs to be implemented in the right order.

There are actually **three different optimization problems**, not one:

Which shipments belong on the same truck?

Then:

In what stop order should that truck serve them?

Then:

Where should each physical piece of freight go inside the trailer?

Those layers must connect.

Imagine:

```
STOP 1 – Milton pickup  
STOP 2 – Mississauga delivery  
STOP 3 – Brampton delivery  
STOP 4 – Vaughan delivery
```

If the trailer is rear-accessed, cargo needed at Mississauga should remain accessible before cargo required only in Vaughan.

So your load planner needs to consider:

geometry

Does it physically fit?

weight

Does total freight remain under capacity?

stackability

Can freight be stacked?

bearing strength

Can this object support another?

orientation

Can it rotate?

delivery order

Will early-stop freight be blocked?

weight distribution

Is placement reasonable across the trailer?

And this is why `hschneid/xflp` is so interesting. Unlike generic “fit boxes in a box” repos, `xflp` explicitly supports truck-loading constraints including loading/unloading locations, LIFO conditions, maximum weight, item bearing weight, stacking groups and a two-axle permissible-load constraint. ²³

Its changelog even describes an unloading-location sequence feature intended for vehicle-routing use cases. ²³

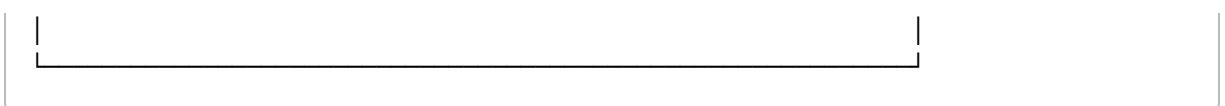
That is extremely close to your idea.

The architecture could be:

```
Dispatch optimiser
  ↓
Truck gets three LTL shipments
  ↓
Route optimiser determines stop order
  ↓
Load planner receives:
  trailer dimensions
  shipment items
  unloading stop for each item
  dimensions / weights / stacking rules
  ↓
xflp solver
  ↓
JSON placement coordinates
  ↓
3D browser renderer
```

And visually:

```
┌────────────────── TRAILER ───────────────────┐
│ [STOP 4][STOP 4][STOP 3][STOP 3][STOP 2][STOP 2] → DOORS │
│ [STOP 4][STOP 4][STOP 3][STOP 3][STOP 2][STOP 2]         │
└──────────────────┴──────────────────┘
```



Click **Play Unload Sequence** and the first-stop pallets slide out first.

That could be extremely memorable in a live demo.

But there is an important RoadStar-data limitation

Your current workbook contains:

- total load weight;
- pallet count;
- trailer length;
- trailer interior height;
- trailer width;
- trailer capacity.

It **does not appear to contain per-pallet/package length, width, height, individual weight, fragility or stacking characteristics**.

Therefore you cannot honestly claim to calculate a physically optimal 3D plan from the supplied dataset alone.

The right approach is:

“RoadStar's current data supports capacity-level planning; our 3D module demonstrates the next layer using enriched pallet-level dimensions. In production those dimensions can be supplied by the shipper/WMS or captured at booking.”

For the demo, clearly label enriched package dimensions as simulated/demo data.

Do **not** pretend a pallet count of 18 tells you enough to determine an actual 3D cargo plan.

That honesty will make the project more credible, not less.

A second useful 3D repo

`skjolber/3d-bin-container-packing` is also very strong. It supports configurable packing controls, obstacles, weight constraints and includes a simple Three.js visualizer. ²⁴

My decision tree would be:

Team comfortable with Java?

Use **xflip** because the trucking constraints are superior.

Need a more flexible generic packer/visualizer?

Use **skjolber**.

Need something functioning in Python tonight?

Use `enzoruiz/3dbinpacking`, get a 3D proof of concept working, then improve constraints later. 25

But again:

3D is allowed to be unfinished. Auto-dispatch is not.

The build plan from now to code freeze

You effectively need to run this like a tiny product company.

Every stage needs a **definition of done**. Do not progress simply because “we worked on it.”

The official schedule gives you remote coding through this week, final preparation on September 12, and a September 13 2 p.m. code freeze. 3

Today: foundation and manual dispatcher

Goal: a dispatcher can actually use the system without AI.

Build:

`Excel → importer → PostgreSQL`

Normalize loads, drivers, trucks, trailers and historical legs.

Then build the shell:

```
Dashboard
Dispatch
Loads
Drivers
Assets
Map
Detention
Analytics
```

Then:

- unassigned loads table;
- drivers/trucks table;
- load detail drawer;

- driver detail/HOS drawer;
- map;
- manual assignment;
- unassign;
- reassignment;
- assignment persistence.

Do not work on 3D yet.

Definition of done:

A team member who did not write the feature can open the site, pick a load, inspect eligible trucks, assign it manually and see that assignment reflected everywhere.

If this does not work, do not move on.

Buffer day: the Morning Auto-Plan

This is your most important development day.

Build candidate generation first.

For every load, produce:

Eligible:

D001

D004

D012

Rejected:

D002 – HOS

D003 – reefer mismatch

D005 – cannot make pickup window

D006 – overweight

Then calculate travel times.

Mapbox's Matrix API can return travel-time/distance matrices for multiple origins and destinations specifically for optimization use cases; if you rely on it, be mindful of its documented coordinate/request limits and batch as necessary. ³⁶

Then OR-Tools chooses assignments globally.

Add:

Plan Morning

Apply Plan

Undo Plan

Why?

Definition of done:

Given a group of unassigned loads and available resources, the system produces a feasible multi-load plan, visibly rejects incompatible assignments and lets the dispatcher apply or override the results.

At that point, you already have a viable hackathon project.

Next: simulation, HOS, geofencing and detention

Build the independent simulator.

Truck moves.

Dashboard updates.

Breadcrumb appears.

Speed/distance accumulates.

HOS clock changes.

Truck enters a facility geofence.

Database saves arrival.

Truck waits.

Detention timer changes.

Truck leaves.

Database saves departure.

Definition of done:

One accelerated end-to-end trip can run unattended from dispatch through pickup, travel, delivery and completed detention accounting.

That hits a huge portion of the project brief in one scenario.  

Then: exceptions and re-optimization

Introduce an event after dispatch.

For example:

401 closure.

Ontario 511 already exposes current events and road conditions that can feed this screen. 

Recalculate ETA.

Then:

```
Original:
ETA 11:16
HOS margin 1h06

Updated:
ETA 11:49
Delivery deadline 11:30
HOS margin 33m

Status: AT RISK

Recommended action:
Re-route via 407
or
Reassign downstream load
```

You don't necessarily need to fully automate every rescue action.

A high-quality recommendation is enough.

Definition of done:

A live event changes operational state and the application responds intelligently rather than merely changing a marker on the map.

Then: driver PWA and integrations

The brief accepts a mobile app **or responsive web view** for the driver interface.  

So do not burn a day creating native iOS/Android applications unless one team member can do that independently.

Make `/driver` responsive/installable.

Driver sees:

TODAY

Milton → London

Pickup 09:30

Delivery 12:00

Dry Van

18 pallets

31,200 lb

[ACCEPT LOAD]

[OPEN ROUTE]

Then status:

En Route to Pickup

Arrived

Loading

En Route

Delivered

Optional:

Upload POD

Dispatcher sees the status immediately.

Final full build day: prove value with the supplied data

This is where you move from “cool student app” to “RoadStar business case.”

Build **Historical Replay**.

Start with a selected historical period.

Display:

ROADSTAR HISTORICAL BASELINE

Loads	...
Loaded km	...
Empty km	...
Empty-distance %	...
HOS risks	...

Then:

Run DispatchOS Simulation

Output:

DISPATCHOS PLAN

Loads covered	...
Projected empty km	...
Projected empty-distance %	...
Late-risk assignments	...
HOS conflicts prevented	...

Do **not** invent savings.

Whatever improvement the solver achieves is what you show.

Even if it only cuts empty travel modestly, real RoadStar data is much stronger evidence than saying:

"AI could save 30%."

The brief explicitly prioritizes workflow speed and financial/operational value, while the official event materials emphasize use of the supplied real industry data and developer resources. [filecite0turn0file0](#)

31

Final prep: only add 3D if everything above is green

At this point:

Core green? Build 3D.

Core unstable? Forget 3D.

If green, parallelize:

1. Create `LoadItem`.
2. Generate/configure demo pallet dimensions.
3. Send items + trailer + stop sequence into xflp.
4. Return positions.
5. Draw simple translucent trailer.
6. Colour/label freight by stop.
7. Add **Play Unloading Sequence**.
8. Show utilization: `Weight 38,200 / 44,500 lb` `Floor/Cube utilization` `Stops 3`
`Unplanned pieces 0`

Do not spend hours making beautiful truck wheels, lighting and photorealistic cardboard.

Boxes inside a transparent trailer are enough.

Judging morning: absolutely no architectural work

The schedule calls for code freeze at **2 p.m. September 13.** 3

On judging day:

- freeze functionality early;
- seed deterministic demo data;
- rehearse;
- test on another laptop;
- test on another network;
- have a local fallback;
- have a pre-recorded simulator path;
- have a backup account;
- have one click that resets the demo.

A slightly less ambitious product that works flawlessly beats a brilliant architecture that crashes halfway through the 401.

The demo I would present

Open with the problem, not the architecture.

"A RoadStar dispatcher starts the day with loads in one system, driver/truck/HOS information in another, and spends hours figuring out what goes where. We built the workflow they already know—but unified—and then automated the decisions that consume their morning."

Scene one: manual process

Show loads.

Show drivers/trucks.

Open one driver.

Open one load.

Manually match them.

Say:

"This is what the dispatcher can already do. Nothing depends on AI."

That directly answers the judge.

Scene two: the moment that should win attention

Undo it.

Select 12 loads.

Press:

Plan My Morning

Assignments appear.

Say:

"Instead of evaluating one truck at a time, DispatchOS evaluates the fleet together."

Open one:

"Why D113?"

Show proximity, HOS, equipment compatibility, pickup slack and future positioning.

Then click a rejected driver:

"Why not D077?"

HOS insufficient.

Now you have demonstrated optimization **and trust**.

Scene three: live operations

Click **Apply Plan**.

Driver PWA gets the load.

Driver accepts.

Truck begins moving.

Map updates.

Breadcrumbs appear.

Scene four: reality breaks the plan

Ontario 511 closure appears.

ETA changes.

System flags delivery/HOS risk.

Recommendation appears.

You approve re-route/re-plan.

Scene five: detention

Truck enters the customer's geofence.

Arrival persisted.

Accelerate simulator.

Two hours pass.

Detention starts.

Departure recorded.

Show billable duration/evidence.

Scene six: deadhead/backhaul

Truck completes London delivery.

System identifies another compatible RoadStar load near London rather than sending it empty.

Show:

Avoided empty travel: X km

using your **actual computed scenario result**.

Scene seven: the visual mic-drop, only if ready

“Dispatch optimization tells us *what freight goes on what trailer*. Our experimental loading planner handles the next layer: *where the freight physically goes*.”

Open transparent trailer.

Three destinations.

Cargo grouped by unload sequence.

Press:

Animate Delivery Order

Pallets for stop one come out without moving stop-three freight.

That is the moment people remember after twelve teams have shown dashboards.

The final product story

The winning pitch is not:

“We built another TMS.”

It is:

“RoadStar already has the data it needs, but the operational state is fragmented across systems and dispatch decisions remain manual. DispatchOS creates one live model of loads, drivers, trucks, trailers, HOS and location. The dispatcher can work exactly as they do today, but with one click the system generates an explainable fleet-wide plan, monitors its execution, captures detention automatically, and continuously re-evaluates the plan as traffic, docks and HOS change.”

And the roadmap is coherent:

```

TODAY
Manual dispatch
    ↓
HACKATHON MVP
Automated matching
    ↓
NEXT
Network-aware multi-load planning
    ↓
NEXT
Dynamic re-dispatch
    ↓
NEXT
Load-board sourcing/backhaul agents
    ↓
NEXT
LTL consolidation
    ↓
NEXT
Physical 3D load planning

```

That mirrors where the commercial industry itself is heading—from unified visibility toward explainable decision automation and ultimately controlled autonomy—without pretending you invented the underlying category. ³⁸

The most important scope decision is therefore very clear:

Build the dispatcher’s morning first. Automate it second. Make it react to the real world third. Add 3D and AI spectacle only after those three layers work.

That approach simultaneously addresses the judge’s explicit feedback, the required deliverables in RoadStar’s brief, the supplied dataset, and the competitive direction of current freight technology.

filecite turn0file0

¹ ¹⁰ <https://www.optimaldynamics.com/resource/optimal-dynamics-launches-scale-the-first-decision-native-agentic-system-tm-built-to-proactively-secure-the-right-freight>

<https://www.optimaldynamics.com/resource/optimal-dynamics-launches-scale-the-first-decision-native-agentic-system-tm-built-to-proactively-secure-the-right-freight>

² ⁵ ¹³ <https://www.mcleodsoftware.com/why-mcleod/resources/press-releases/now-available-samsara-enhanced-workflow-integration-with-mcleod-softwares-loadmaster-enterprise/>

<https://www.mcleodsoftware.com/why-mcleod/resources/press-releases/now-available-samsara-enhanced-workflow-integration-with-mcleod-softwares-loadmaster-enterprise/>

³ <https://www.roadstarhackathon.com/schedule>

<https://www.roadstarhackathon.com/schedule>

- 4 <https://www.samsara.com/ca/products/telematics>
<https://www.samsara.com/ca/products/telematics>
- 6 <https://alvys.com/>
<https://alvys.com/>
- 7 <https://www.samsara.com/resources/marketplace/alvys-inc>
<https://www.samsara.com/resources/marketplace/alvys-inc>
- 8 <https://www.optimaldynamics.com/platform/decision-automation>
<https://www.optimaldynamics.com/platform/decision-automation>
- 9 <https://support.optimaldynamics.com/hc/en-us/articles/36396414595981-What-Is-Atlas>
<https://support.optimaldynamics.com/hc/en-us/articles/36396414595981-What-Is-Atlas>
- 11 <https://helpcenter.gomotive.com/hc/en-us/articles/31852213827357-How-to-enable-TorqueAI-Dispatch-integration>
<https://helpcenter.gomotive.com/hc/en-us/articles/31852213827357-How-to-enable-TorqueAI-Dispatch-integration>
- 12 <https://kb.samsara.com/hc/en-us/articles/4423183155341-Get-Started-with-the-Samsara-Driver-App/>
<https://kb.samsara.com/hc/en-us/articles/4423183155341-Get-Started-with-the-Samsara-Driver-App/>
- 14 38 <https://alvys.com/features/alvys-intelligence>
<https://alvys.com/features/alvys-intelligence>
- 15 <https://www.optimaldynamics.com/resource/optimal-dynamics-enters-canadian-market-through-strategic-partnership-with-challenger-motor-freight>
<https://www.optimaldynamics.com/resource/optimal-dynamics-enters-canadian-market-through-strategic-partnership-with-challenger-motor-freight>
- 16 34 37 <https://511on.ca/developers/doc>
<https://511on.ca/developers/doc>
- 17 30 <https://github.com/VROOM-Project/vroom>
<https://github.com/VROOM-Project/vroom>
- 18 <https://github.com/traccar/traccar/>
<https://github.com/traccar/traccar/>
- 19 <https://github.com/traccar/traccar-web>
<https://github.com/traccar/traccar-web>
- 20 <https://github.com/maplibre/maplibre-gl-js>
<https://github.com/maplibre/maplibre-gl-js>
- 21 <https://github.com/Turfjs/turf>
<https://github.com/Turfjs/turf>
- 22 <https://github.com/graphhopper/graphhopper>
<https://github.com/graphhopper/graphhopper>
- 23 <https://github.com/hschneid/xflp>
<https://github.com/hschneid/xflp>
- 24 <https://github.com/skjolber/3d-bin-container-packing>
<https://github.com/skjolber/3d-bin-container-packing>

25 <https://github.com/enzorui/3dbinpacking>

<https://github.com/enzorui/3dbinpacking>

26 <https://github.com/timefoldai/timefold-solver>

<https://github.com/timefoldai/timefold-solver>

27 <https://github.com/fleetbase/fleetbase>

<https://github.com/fleetbase/fleetbase>

28 <https://github.com/suxrobGM/logistics-app>

<https://github.com/suxrobGM/logistics-app>

29 <https://developers.google.com/optimization/routing/vrptw>

<https://developers.google.com/optimization/routing/vrptw>

31 <https://www.roadstarhackathon.com/>

<https://www.roadstarhackathon.com/>

32 **Commercial Vehicle Drivers Hours of Service Regulations**

https://laws-lois.justice.gc.ca/eng/regulations/SOR-2005-313/section-13.html?utm_source=chatgpt.com

33 <https://www.samsara.com/ca/support/developers>

<https://www.samsara.com/ca/support/developers>

35 <https://511on.ca/help/endpoint/cameras>

<https://511on.ca/help/endpoint/cameras>

36 <https://docs.mapbox.com/api/navigation/matrix/>

<https://docs.mapbox.com/api/navigation/matrix/>